

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: S Satya Jayavanth

Roll No.: A24126552116

Date: 29-08-2026

1. TITLE

Write a program using JavaScript (arrays, functions)

2. AIM

To write and execute a JavaScript program to understand and demonstrate the use of **arrays and functions**, including array methods, operators, higher-order methods, different types of functions and function invocation.

3. OBJECTIVE

The objectives of this experiment are:

- ☐ To understand how arrays are created and initialized in JavaScript.
 - ☐ To access and modify elements of an array.
 - ☐ To understand the length property of an array.
 - ☐ To perform array operations using mutable and immutable methods.
 - ☐ To understand the use of spread and rest operators with arrays.
 - ☐ To use higher-order array methods such as `forEach()` and `map()`.
 - ☐ To understand different types of JavaScript functions.
 - ☐ To implement function declaration, function expression and arrow functions.
 - ☐ To pass parameters to functions and return values.
 - ☐ To create a master function that calls other functions based on the selected operation.
-

4. THEORY

An **array** in JavaScript is a data structure used to store multiple values in a single variable. JavaScript arrays can contain different types of values such as strings, numbers, Boolean values and objects. Arrays can be created using array literals or the Array constructor. JavaScript provides several built-in methods for manipulating arrays. Methods such as `push()`, `pop()`, `unshift()`, `shift()` and `splice()` can modify the original array. Methods such as `slice()` and `concat()` can create new arrays without directly modifying the original array. Other methods such as `indexOf()` and `includes()` can be used to search for elements. The **spread operator (...)** allows the elements of an array to be expanded into another array. JavaScript also provides higher-order array methods such as `forEach()` and `map()`. The `forEach()` method executes a function for each element, while `map()` creates a new array by applying a function to every element. A **function** is a reusable block of code designed to perform a particular task. JavaScript supports different ways of defining functions. A function declaration uses the function keyword, a function expression stores a function inside a variable, and an arrow function provides a shorter syntax for writing functions. In this experiment, functions are used to perform addition, subtraction, multiplication and division. A master `calculate()` function accepts the operation and operands and calls the appropriate function using a switch statement.

5. ALGORITHM / PROCEDURE

A. Arrays

1. Create an array using an array literal containing different data types.
2. Create another array using the Array constructor.
3. Display the arrays using `console.log()`.
4. Create an array containing fruit names.
5. Access an array element using its index.
6. Modify an array element using its index.
7. Display the length of the array using the `length` property.
8. Add an element at the end using `push()`.
9. Remove the last element using `pop()`.
10. Add an element at the beginning using `unshift()`.
11. Remove the first element using `shift()`.
12. Replace an element using `splice()`.
13. Extract elements using `slice()`.
14. Combine arrays using `concat()`.
15. Find the position of an element using `indexOf()`.
16. Check whether an element exists using `includes()`.
17. Use the spread operator to clone and extend an array.
18. Use `forEach()` to perform an operation for every element.
19. Use `map()` to create a new array containing doubled values.
20. Display the results using `console.log()`.

B. Functions

1. Define an add() function using a standard function declaration.
 2. Define a subtract() function using a function expression.
 3. Define a multiply() function using an arrow function.
 4. Define a divide() arrow function with a condition to check division by zero.
 5. Create a calculate() master function.
 6. Pass the operation name and two numbers to the calculate() function.
 7. Use a switch statement to select the required operation.
 8. Call the corresponding function based on the operation.
 9. Display the calculated result.
 10. Test addition, subtraction, multiplication and division with different values.
 11. Test division by zero and display an appropriate error message.
-

6. PROGRAM

Arrays.js

```
//normal initialize
const items = ["Apple", 42, true, { id: 1 }];
//constructor
const numbers = new Array(1, 2, 3);
//printing
console.log(items)

console.log(numbers)

const fruits = ["Apple", "Banana", "Cherry"];
//access
console.log(fruits[0]);
//modify
fruits[1] = "Mango";
//length
console.log(fruits.length);
//methods(mutable)
fruits.push("Kiwi");
console.log(fruits);
numbers.pop();
console.log(numbers)
fruits.unshift("Sapota");
console.log(fruits);
items.shift();
console.log(items)
fruits.splice(1,1,"Pear")
console.log(fruits);
```

```

// (un mutable)
console.log(fruits.slice(2,4));
console.log(items.concat(numbers));
console.log(fruits.indexOf('Mango'));
console.log(numbers.includes(2));
// spread and rest operators
const original =[1,2];
const cloneAndExtend = [...original, 3, 4];
console.log(cloneAndExtend);
// higher methods
const prices =[1,2,3,4,5,6];
prices.forEach(price => console.log(`Price: ${price}`));
const numberssss =[1,2,3,4,5,6];
const doubled = numberssss.map(num => num * 2);
console.log(doubled);

```

functions.js

```

// =====
// 1. FUNCTION DECLARATION (Standard)
// =====
// This function takes two parameters and returns their sum
function add(num1, num2) {
    return num1 + num2;
}

// =====
// 2. FUNCTION EXPRESSION
// =====
// This anonymous function is stored inside a variable
const subtract = function(num1, num2) {
    return num1 - num2;
};

// =====
// 3. ARROW FUNCTION (Modern ES6+)
// =====
// A shorter way to write clean, concise functions
const multiply = (num1, num2) => num1 * num2;
const divide = (num1, num2) => {
    if (num2 === 0) {
        return "Error: Cannot divide by zero!";
    }
    return num1 / num2;
}

```

```

};

// =====
// 4. MASTER FUNCTION (Calling other functions)
// =====
// This function handles the program flow by accepting an operation name
function calculate(operation, a, b) {
  console.log(`--- Calculating ${operation} with ${a} and ${b} ---`);

  let result;

  switch (operation.toLowerCase()) {
    case 'add':
      result = add(a, b); // Calling the declaration function
      break;
    case 'subtract':
      result = subtract(a, b); // Calling the function expression
      break;
    case 'multiply':
      result = multiply(a, b); // Calling the arrow function
      break;
    case 'divide':
      result = divide(a, b); // Calling the arrow function
      break;
    default:
      result = "Invalid Operation!";
  }

  console.log(`Result: ${result}\n`);
}

// =====
// CALLING / INVOKING THE FUNCTIONS
// =====
calculate('add', 10, 5);    // Output: Result: 15
calculate('subtract', 20, 8); // Output: Result: 12
calculate('multiply', 4, 3); // Output: Result: 12
calculate('divide', 10, 2);  // Output: Result: 5
calculate('divide', 7, 0);   // Output: Result: Error: Cannot divide by zero!

```

7. CODE EXPLANATION

Array Program

const items = ["Apple", 42, true, { id: 1 }];

Creates an array containing different data types such as a string, number, Boolean and object.

const numbers = new Array(1, 2, 3);

Creates an array using the Array constructor.

fruits[0]

Accesses the first element of the fruits array using its index.

fruits[1] = "Mango";

Modifies the second element of the array.

fruits.length

Returns the number of elements present in the array.

push()

Adds an element to the end of an array.

pop()

Removes the last element from an array.

unshift()

Adds an element to the beginning of an array.

shift()

Removes the first element from an array.

splice()

Adds, removes or replaces elements at a specified position in an array.

slice()

Extracts a portion of an array and returns it as a new array.

concat()

Combines two or more arrays and returns a new array.

indexOf()

Returns the index position of a specified element.

includes()

Checks whether a specified element exists in an array.

... (Spread Operator)

Expands the elements of an array and is used in the program to create a clone and extend it with additional values.

forEach()

Executes the specified function once for each element in the array.

map()

Creates a new array by applying the specified operation to each element.

Functions Program

function add(num1, num2)

Defines a standard function that returns the sum of two numbers.

const subtract = function(num1, num2)

Defines a function expression and stores it in the subtract variable.

const multiply = (num1, num2) => num1 * num2

Defines an arrow function that returns the product of two numbers.

divide()

Performs division and checks whether the second number is zero before performing the operation.

calculate(operation, a, b)

Acts as the master function and controls the execution of the required arithmetic operation.

switch(operation.toLowerCase())

Checks the operation name and selects the corresponding function.

return

Returns the calculated value from a function.

console.log()

Displays the output in the browser's JavaScript console.

8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Create and print items array	Array containing different data types should be displayed	Array displayed	Pass
TC-02	Access fruits[0]	Apple should be displayed	Apple displayed	Pass
TC-03	Modify fruits[1] to Mango	Second element should become Mango	Mango displayed	Pass
TC-04	Apply push() and pop()	Elements should be added/removed correctly	Operations performed correctly	Pass
TC-05	Apply unshift() and shift()	First element should be added/removed correctly	Operations performed correctly	Pass

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-06	Apply splice()	Specified element should be replaced	Element replaced successfully	Pass
TC-07	Apply slice() and concat()	Required portion/combined array should be returned	Correct result displayed	Pass
TC-08	Apply indexOf() and includes()	Element position/existence should be determined	Correct result displayed	Pass
TC-09	Apply map() to [1,2,3,4,5,6]	Array should become [2,4,6,8,10,12]	Correct result displayed	Pass
TC-10	Call calculate('add', 10, 5)	Result should be 15	15 displayed	Pass
TC-11	Call calculate('subtract', 20, 8)	Result should be 12	12 displayed	Pass
TC-12	Call calculate('multiply', 4, 3)	Result should be 12	12 displayed	Pass
TC-13	Call calculate('divide', 10, 2)	Result should be 5	5 displayed	Pass
TC-14	Call calculate('divide', 7, 0)	Division-by-zero error message should be displayed	Error message displayed	Pass

9. OUTPUT

Arrays.js

```
PS C:\Users\satya jayavanth\OneDrive\Desktop\FS prac> node rec.js
[ 'Apple', 42, true, { id: 1 } ]
[ 1, 2, 3 ]
Apple
3
[ 'Apple', 'Mango', 'Cherry', 'Kiwi' ]
[ 1, 2 ]
[ 'Sapota', 'Apple', 'Mango', 'Cherry', 'Kiwi' ]
[ 42, true, { id: 1 } ]
[ 'Sapota', 'Pear', 'Mango', 'Cherry', 'Kiwi' ]
[ 'Mango', 'Cherry' ]
[ 42, true, { id: 1 }, 1, 2 ]
2
true
[ 1, 2, 3, 4 ]
Price: $1
Price: $2
Price: $3
Price: $4
Price: $5
Price: $6
[ 2, 4, 6, 8, 10, 12 ]
```

Functions.js

```
PS C:\Users\satya jayavanth\OneDrive\Desktop\FS prac> node rec.js
--- Calculating add with 10 and 5 ---
Result: 15

--- Calculating subtract with 20 and 8 ---
Result: 12

--- Calculating multiply with 4 and 3 ---
Result: 12

--- Calculating divide with 10 and 2 ---
Result: 5

--- Calculating divide with 7 and 0 ---
Result: Error: Cannot divide by zero!
```

10. RESULT

Thus, a structured static webpage was successfully designed and developed using HTML5 semantic elements and HTML components. The webpage was verified for its structure, navigation links, form controls, lists, sidebar and footer.